■ AutoDeploy

Production-Grade CI/CD Pipeline

Docker · Minikube · GitHub Actions · Prometheus · Grafana

COMPLETE BUILD GUIDE + ALL PROJECT FILES + AI PROMPTS

Pravinkumar Choudhary

BCA · Cloud Computing · Parul University · github.com/speedprav

14 Days	100% Free	6 Tech Tools	Resume Ready
Build Plan	Zero Cloud Cost	Full Stack DevOps	+ Interview Q&A;

■ Table of Contents

SECTION 1 — Project Overview & Architecture

What you are building, why it matters, and how it all connects

What is AutoDeploy?

AutoDeploy is a fully automated DevOps pipeline that mimics exactly what professional DevOps engineers build at real tech companies. Every time you push code to GitHub, a 4-stage pipeline automatically tests your code, packages it into a Docker container, pushes it to a registry, and deploys it to a Kubernetes cluster — all without any manual intervention. A monitoring stack then tracks your app health in real time.

Architecture Flow

Stage	Trigger	Tool	Output
1. Code Push	git push origin main	GitHub	Pipeline starts
2. Test	Automatic	GitHub Actions + pytest	Pass / Fail
3. Build	Tests pass	Docker	Container image
4. Push	Build succeeds	Docker Hub	Image stored
5. Deploy	Push succeeds	kubectl + Minikube	New version live
6. Monitor	Always on	Prometheus + Grafana	Live dashboard

Free Tech Stack

Layer	Tool	Version	Purpose	Cost
Web App	Python FastAPI	0.104+	REST API web server	Free
Testing	pytest	7.4+	Automated test runner	Free
Container	Docker Desktop	Latest	Build & run containers	Free
Registry	Docker Hub	—	Store Docker images	Free
Kubernetes	Minikube	1.32+	Local K8s cluster	Free
K8s CLI	kubectl	1.29+	Control Kubernetes	Free
CI/CD	GitHub Actions	—	Automate pipeline	Free
Pkg Mgr	Helm	3.x	Install K8s apps	Free
Metrics	Prometheus	2.x	Collect metrics	Free
Dashboard	Grafana	10.x	Visualize metrics	Free

Project Folder Structure (Final)

■ Project Tree

```
autodeploy/
■■■ app/
■ ■■■ main.py # FastAPI application
■ ■■■ requirements.txt # Python dependencies
■ ■■■ tests/
■ ■■■ test_main.py # Pytest test suite
```

```
■■■ k8s/
■ ■■■ deployment.yaml # Kubernetes deployment
■ ■■■ service.yaml # Kubernetes service
■ ■■■ ingress.yaml # Kubernetes ingress
■ ■■■ servicemonitor.yaml # Prometheus scrape config
■■■ monitoring/
■ ■■■ prometheus-values.yaml # Helm values for Prometheus stack
■■■ .github/
■ ■■■ workflows/
■ ■■■ deploy.yml # GitHub Actions CI/CD pipeline
■■■ Dockerfile # Container build instructions
■■■ docker-compose.yml # Local dev environment
■■■ .dockerignore # Files excluded from Docker build
■■■ README.md # Project documentation
```

SECTION 2 — Prerequisites & Environment Setup

Install and verify every tool before writing a single line of code

■■ *Complete ALL steps in this section before moving to Day 1. Skipping setup causes mysterious errors later that waste hours.*

Step 1 — Install Docker Desktop

Docker Desktop runs containers on your local machine. It also includes the Docker CLI.

```
# Download from: https://www.docker.com/products/docker-desktop
# After installing, open Docker Desktop and wait for it to say "Engine Running"
# Verify installation:
docker --version
docker run hello-world
# Expected output: "Hello from Docker!"
# This confirms Docker is working correctly.
```

Step 2 — Install Minikube

Minikube creates a local single-node Kubernetes cluster on your machine.

```
# Windows (PowerShell as Admin):
winget install Kubernetes.minikube
# Mac:
brew install minikube
# Linux:
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
# Verify:
minikube version
# Expected: minikube version: v1.32.x
```

Step 3 — Install kubectl

```
# Windows (PowerShell as Admin):
winget install Kubernetes.kubectl
# Mac:
brew install kubectl
# Linux:
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
# Verify:
kubectl version --client
# Expected: Client Version: v1.29.x
```

Step 4 — Install Python 3.10+

```
# Download from: https://www.python.org/downloads/
# During installation on Windows: CHECK "Add Python to PATH"
# Verify:
python --version
pip --version
# Expected: Python 3.10.x or higher
```

Step 5 — Install Helm

```
# Windows (PowerShell as Admin):
winget install Helm.Helm
# Mac:
brew install helm
# Linux:
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
# Verify:
helm version
# Expected: version.BuildInfo{Version:"v3.x.x"}
```

Step 6 — GitHub Repository Setup

1 **Create GitHub Repository**

Go to github.com → New Repository → Name: autodeploy → Public → Create

2 **Clone to your machine**

git clone https://github.com/YOUR_USERNAME/autodeploy.git && cd autodeploy

3 **Create Docker Hub account**

Go to hub.docker.com → Sign Up → Remember your username (used throughout project)

4 **Create Docker Hub Access Token**

Docker Hub → Account Settings → Security → New Access Token → Copy it (used in GitHub Secrets)

- ■ docker --version returns a version number
- ■ docker run hello-world prints Hello from Docker
- ■ minikube version returns a version number
- ■ kubectl version --client returns a version number
- ■ python --version returns 3.10 or higher
- ■ helm version returns a version number
- ■ GitHub repository created and cloned locally
- ■ Docker Hub account created with access token saved

SECTION 3 — Days 1–2: FastAPI App + Docker

Build your Python web application and containerize it

Goal: By end of Day 2, you have a Python web app running inside a Docker container, pushed to Docker Hub, and committed to GitHub. Every file below is complete — just copy it.

■ AI PROMPT — Day 1 — If you want AI to generate this code

I am building a DevOps portfolio project. Generate a Python FastAPI app with these exact endpoints: GET /health returns status+timestamp, GET /info returns project metadata, POST /predict accepts {text: string} and returns a mock NLP analysis. Include complete main.py, requirements.txt, and tests/test_main.py with pytest. Add detailed inline comments explaining every line.

3.1 — Complete main.py

■ app/main.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from datetime import datetime
import uvicorn
import random
# Initialize the FastAPI application
app = FastAPI(
title="AutoDeploy API",
description="Production CI/CD demo app for DevOps portfolio",
version="1.0.0"
)
# ■■ Data Models ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class PredictRequest(BaseModel):
"""Request body for the /predict endpoint"""
text: str
class PredictResponse(BaseModel):
"""Response body for the /predict endpoint"""
input_text: str
word_count: int
sentiment: str
confidence: float
processed_at: str
# ■■ Endpoints
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@app.get("/health")
def health_check():
"""
Health check endpoint — Kubernetes uses this to verify the app is alive.
Returns: status and current UTC timestamp
```

```
"""
return {
"status": "healthy",
"timestamp": datetime.utcnow().isoformat(),
"service": "autodeploy-api"
}
@app.get("/info")
def get_info():
"""
Application metadata endpoint.
Returns: project details and author information
"""
return {
"project": "AutoDeploy",
"version": "1.0.0",
"author": "Pravinkumar Choudhary",
"github": "github.com/speedprav",
"description": "Production CI/CD pipeline demo",
"tech_stack": [
"FastAPI", "Docker", "Kubernetes",
"GitHub Actions", "Prometheus", "Grafana"
]
}
@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest):
"""
Mock NLP prediction endpoint.
Accepts text input and returns a simple sentiment analysis result.
In a real app, this would call an ML model.
"""
if not request.text.strip():
raise HTTPException(
status_code=400,
detail="Text field cannot be empty"
)
# Count words in the input text
word_count = len(request.text.split())
# Simple mock sentiment: positive words trigger positive sentiment
positive_words = ["good", "great", "excellent", "happy", "love",
"best", "amazing", "wonderful", "fantastic"]
negative_words = ["bad", "terrible", "awful", "hate", "worst",
"horrible", "poor", "disappointing"]
text_lower = request.text.lower()
pos_count = sum(1 for w in positive_words if w in text_lower)
neg_count = sum(1 for w in negative_words if w in text_lower)
if pos_count > neg_count:
sentiment = "positive"
confidence = round(random.uniform(0.75, 0.95), 2)
elif neg_count > pos_count:
sentiment = "negative"
```

```
confidence = round(random.uniform(0.70, 0.90), 2)
else:
sentiment = "neutral"
confidence = round(random.uniform(0.60, 0.80), 2)
return PredictResponse(
input_text=request.text,
word_count=word_count,
sentiment=sentiment,
confidence=confidence,
processed_at=datetime.utcnow().isoformat()
)
# ■■ Run locally
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
if __name__ == "__main__":
uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)
```

3.2 — Complete requirements.txt

■ app/requirements.txt

```
fastapi==0.104.1
uvicorn[standard]==0.24.0
pydantic==2.5.0
pytest==7.4.3
httpx==0.25.2
prometheus-fastapi-instrumentator==6.1.0
```

3.3 — Complete test_main.py

■ app/tests/test_main.py

```
import pytest
from fastapi.testclient import TestClient
import sys
import os
# Add parent directory to path so we can import main
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from main import app
# Create a test client — this simulates HTTP requests without running a server
client = TestClient(app)
# ■■ Health Endpoint Tests ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def test_health_returns_200():
"""Health endpoint must return HTTP 200 OK"""
response = client.get("/health")
assert response.status_code == 200
def test_health_returns_healthy_status():
"""Health endpoint must return status: healthy"""
response = client.get("/health")
data = response.json()
```

```
assert data["status"] == "healthy"
def test_health_has_timestamp():
"""Health endpoint must include a timestamp"""
response = client.get("/health")
data = response.json()
assert "timestamp" in data
# ■■ Info Endpoint Tests ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def test_info_returns_200():
"""Info endpoint must return HTTP 200 OK"""
response = client.get("/info")
assert response.status_code == 200
def test_info_has_required_fields():
"""Info endpoint must return project, version, and author"""
response = client.get("/info")
data = response.json()
assert "project" in data
assert "version" in data
assert "author" in data
def test_info_correct_project_name():
"""Info endpoint must return correct project name"""
response = client.get("/info")
data = response.json()
assert data["project"] == "AutoDeploy"
# ■■ Predict Endpoint Tests ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def test_predict_returns_200():
"""Predict endpoint must return HTTP 200 for valid input"""
response = client.post("/predict", json={"text": "This is a great product!"})
assert response.status_code == 200
def test_predict_positive_sentiment():
"""Predict endpoint should return positive for clearly positive text"""
response = client.post("/predict", json={"text": "I love this amazing product"})
data = response.json()
assert data["sentiment"] == "positive"
def test_predict_negative_sentiment():
"""Predict endpoint should return negative for clearly negative text"""
response = client.post("/predict", json={"text": "This is terrible and awful"})
data = response.json()
assert data["sentiment"] == "negative"
def test_predict_returns_word_count():
"""Predict endpoint must return correct word count"""
response = client.post("/predict", json={"text": "one two three"})
data = response.json()
assert data["word_count"] == 3
def test_predict_empty_text_returns_400():
"""Predict endpoint must reject empty text with 400 error"""
response = client.post("/predict", json={"text": ""})
assert response.status_code == 400
def test_predict_has_confidence_score():
"""Predict endpoint must return a confidence score between 0 and 1"""
```

```
response = client.post("/predict", json={"text": "This is good"})
data = response.json()
assert 0.0 <= data["confidence"] <= 1.0
```

3.4 — Complete Dockerfile

■ Dockerfile

```
# ■■ Stage 1: Builder ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Use slim Python image to keep the final image small
FROM python:3.11-slim as builder
# Set working directory inside the container
WORKDIR /app
# Copy only requirements first (Docker caches this layer if requirements don't change)
# This makes subsequent builds much faster
COPY app/requirements.txt .
# Install dependencies into a separate folder for copying later
RUN pip install --user --no-cache-dir -r requirements.txt
# ■■ Stage 2: Production image ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
FROM python:3.11-slim
# Create a non-root user for security (never run production apps as root)
RUN useradd --create-home --shell /bin/bash appuser
WORKDIR /app
# Copy installed packages from builder stage
COPY --from=builder /root/.local /home/appuser/.local
# Copy application code
COPY app/ .
# Set ownership to non-root user
RUN chown -R appuser:appuser /app
# Switch to non-root user
USER appuser
# Add local Python packages to PATH
ENV PATH=/home/appuser/.local/bin:$PATH
# Expose port 8000 (FastAPI default)
EXPOSE 8000
# Health check — Docker will run this every 30s to verify the container is healthy
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"
# Start the application
# --host 0.0.0.0 makes it accessible from outside the container
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

3.5 — Complete docker-compose.yml

■ docker-compose.yml

```
version: '3.8'
services:
```

```
api:
# Build the image from our Dockerfile
build:
context: .
dockerfile: Dockerfile
# Map host port 8000 to container port 8000
ports:
- "8000:8000"
# Mount app folder for live code reloading during development
volumes:
- ./app:/app
# Environment variables
environment:
- ENV=development
# Restart policy: always restart unless manually stopped
restart: unless-stopped
# Health check configuration
healthcheck:
test: ["CMD", "python", "-c",
"import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"]
interval: 30s
timeout: 10s
retries: 3
start_period: 10s
```

3.6 — Complete .dockerignore

■ .dockerignore

```
# Python cache files
__pycache__/
*.py[cod]
*$py.class
*.pyc
# Virtual environments
.env
.venv
env/
venv/
# Test artifacts
.pytest_cache/
.coverage
htmlcov/
# IDE files
.vscode/
.idea/
*.swp
# Git
.git/
```

```
.gitignore
# Docker files (don't include in image)
Dockerfile
docker-compose.yml
.dockerignore
# Documentation
README.md
docs/
# Kubernetes manifests
k8s/
# CI/CD
.github/
```

3.7 — Step-by-Step Day 1 & 2 Build Guide

1 **Create project folder structure**

mkdir -p autodeploy/app/tests autodeploy/k8s autodeploy/monitoring autodeploy/.github/workflows

2 **Create all files**

Copy every file from Section 3.1 to 3.6 into the correct paths shown in the filename header

3 **Create Python virtual environment**

cd autodeploy && python -m venv venv && source venv/bin/activate (Windows: venv\Scripts\activate)

4 **Install dependencies**

pip install -r app/requirements.txt

5 **Run tests locally first**

cd app && pytest tests/ -v — All 11 tests should pass

6 **Run app locally**

uvicorn main:app --reload — Open http://localhost:8000/health in browser

7 **Build Docker image**

docker build -t YOUR_DOCKERHUB_USERNAME/autodeploy:latest .

8 **Run container locally**

docker run -p 8000:8000 YOUR_DOCKERHUB_USERNAME/autodeploy:latest

9 **Test the running container**

Open http://localhost:8000/health — should return {status: healthy}

10 **Login and push to Docker Hub**

docker login && docker push YOUR_DOCKERHUB_USERNAME/autodeploy:latest

11 **Commit to GitHub**

git add . && git commit -m 'Day 2: FastAPI app + Docker' && git push

■ pytest tests/ -v shows 11 passed

■ http://localhost:8000/health returns {status: healthy}

■ http://localhost:8000/info returns project metadata

■ docker ps shows your container running

■ Docker Hub shows your image at hub.docker.com/r/YOUR_USERNAME/autodeploy

■ GitHub shows all files committed

SECTION 4 — Days 3–4: Kubernetes with Minikube

Deploy your container to a local Kubernetes cluster and learn K8s operations

Goal: By end of Day 4, your app runs inside Minikube with 2 replicas, you can trigger a rolling update, verify zero-downtime deployment, and roll back if needed.

■ AI PROMPT — Day 3 — Kubernetes Manifests

I have a Docker image YOUR_USERNAME/autodeploy:latest on Docker Hub. Generate: deployment.yaml with 2 replicas and resource limits, service.yaml NodePort, and ingress.yaml. Add liveness and readiness probes pointing to /health. Explain every field in the YAML with inline comments.

4.1 — Complete deployment.yaml

■ k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: autodeploy # Name of this deployment
labels:
app: autodeploy # Label used to identify related resources
spec:
replicas: 2 # Run 2 identical pods for high availability
selector:
matchLabels:
app: autodeploy # This deployment manages pods with this label
strategy:
type: RollingUpdate # Update pods one at a time (zero downtime)
rollingUpdate:
maxSurge: 1 # Allow 1 extra pod during update
maxUnavailable: 0 # Never take a pod offline before new one is ready
template:
metadata:
labels:
app: autodeploy # Pods get this label
spec:
containers:
- name: autodeploy
image: YOUR_DOCKERHUB_USERNAME/autodeploy:latest
imagePullPolicy: Always # Always pull latest image
ports:
- containerPort: 8000 # Port your FastAPI app listens on
# Resource limits — prevents one pod from consuming all node resources
resources:
requests:
```

```
memory: "64Mi" # Minimum memory to reserve
cpu: "100m" # 100 millicores = 0.1 CPU core
limits:
memory: "128Mi" # Maximum memory allowed
cpu: "200m" # Maximum CPU allowed
# Liveness probe — if this fails, Kubernetes restarts the pod
livenessProbe:
httpGet:
path: /health # Calls our /health endpoint
port: 8000
initialDelaySeconds: 10 # Wait 10s before first check
periodSeconds: 30 # Check every 30 seconds
failureThreshold: 3 # Restart after 3 consecutive failures
# Readiness probe — if this fails, pod is removed from load balancer
readinessProbe:
httpGet:
path: /health
port: 8000
initialDelaySeconds: 5
periodSeconds: 10
failureThreshold: 3
# Environment variables inside the container
env:
- name: ENV
value: "production"
```

4.2 — Complete service.yaml

■ k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
name: autodeploy-service # Name of this service
spec:
type: NodePort # Exposes the service on each Node's IP at a static port
# This lets us access it from outside the cluster
selector:
app: autodeploy # Route traffic to pods with this label
ports:
- protocol: TCP
port: 80 # Port the service listens on inside the cluster
targetPort: 8000 # Port on the pod to forward traffic to
nodePort: 30080 # External port on the Minikube node (30000-32767)
```

4.3 — Complete ingress.yaml

■ k8s/ingress.yaml

```
apiVersion: networking.k8s.io/v1
```

```
kind: Ingress
metadata:
name: autodeploy-ingress
annotations:
# Use nginx ingress controller
nginx.ingress.kubernetes.io/rewrite-target: /
spec:
rules:
- host: autodeploy.local # Local hostname (add to /etc/hosts)
http:
paths:
- path: /
pathType: Prefix
backend:
service:
name: autodeploy-service
port:
number: 80
```

4.4 — Step-by-Step Kubernetes Guide

1. **Start Minikube cluster**
   minikube start --driver=docker --cpus=2 --memory=4096
2. **Enable Ingress addon**
   minikube addons enable ingress
3. **Edit deployment.yaml**
   Replace YOUR_DOCKERHUB_USERNAME with your actual Docker Hub username in deployment.yaml
4. **Apply all Kubernetes manifests**
   kubectl apply -f k8s/ — This creates deployment, service, and ingress
5. **Watch pods start up**
   kubectl get pods --watch — Wait until STATUS shows Running for both pods
6. **Get the app URL**
   minikube service autodeploy-service --url — Copy this URL and open in browser
7. **Test your app on Kubernetes**
   Open URL/health in browser — should return {status: healthy}
8. **Practice rolling update**
   kubectl set image deployment/autodeploy autodeploy=YOUR_USERNAME/autodeploy:v2
9. **Watch the rollout**
   kubectl rollout status deployment/autodeploy

10 **Practice rollback**

kubectl rollout undo deployment/autodeploy

11 **Practice scaling**

kubectl scale deployment autodeploy --replicas=4 then back to --replicas=2

Key kubectl Commands Cheat Sheet

■ **kubectl Quick Reference**

```
# Cluster info
minikube status # Is Minikube running?
kubectl cluster-info # Cluster details
kubectl get nodes # Node status

# Pod management
kubectl get pods # List all pods
kubectl get pods -o wide # With IP and node info
kubectl describe pod POD_NAME # Full pod details + events
kubectl logs POD_NAME # View pod logs
kubectl logs -f POD_NAME # Follow live logs
kubectl exec -it POD_NAME -- /bin/bash # Shell into pod

# Deployment management
kubectl get deployments # List deployments
kubectl rollout status deployment/autodeploy # Watch rollout progress
kubectl rollout history deployment/autodeploy # See rollout history
kubectl rollout undo deployment/autodeploy # Rollback to previous version

# Service management
kubectl get services # List services
minikube service autodeploy-service --url # Get accessible URL

# Apply / delete
kubectl apply -f k8s/ # Apply all manifests in folder
kubectl delete -f k8s/ # Delete all resources

# Debugging
kubectl get events --sort-by=.metadata.creationTimestamp # Recent events
kubectl describe deployment autodeploy # Deployment details
```

■ minikube status shows Running

■ kubectl get pods shows 2 pods with STATUS: Running

■ Browser URL/health returns {status: healthy}

■ Rolling update completed without downtime

■ kubectl rollout undo successfully rolled back

■ kubectl scale successfully changed replica count

SECTION 5 — Days 5–7: GitHub Actions CI/CD Pipeline

Automate every deployment — zero manual steps from code push to live update

Goal: Every git push to main branch triggers an automatic 4-job pipeline. If any job fails, the pipeline stops and nothing gets deployed. If all jobs pass, new version is live in Kubernetes automatically.

■ AI PROMPT — Day 5 — GitHub Actions Workflow

Generate a complete .github/workflows/deploy.yml with 4 sequential jobs: test (pytest), build (docker build), push (docker push to Docker Hub with :latest and :SHA tags), deploy (kubectl set image). Jobs must use 'needs:' to run in sequence. Use GitHub Secrets DOCKERHUB_USERNAME and DOCKERHUB_TOKEN. Deploy only on main branch push. Add detailed comments explaining every step.

5.1 — Complete deploy.yml — Full CI/CD Pipeline

■ .github/workflows/deploy.yml

```
name: AutoDeploy CI/CD Pipeline
# ■■ Trigger
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
on:
push:
branches: [ main ] # Only run when pushing to main branch
pull_request:
branches: [ main ] # Also run on PRs (test + build only, not deploy)
# ■■ Environment Variables ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
env:
DOCKER_IMAGE: ${{ secrets.DOCKERHUB_USERNAME }}/autodeploy
DEPLOYMENT_NAME: autodeploy
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■
# JOB 1: TEST
# Run automated tests — fail fast before wasting time building
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■
jobs:
test:
name: Run Tests
runs-on: ubuntu-latest
steps:
- name: Checkout code
uses: actions/checkout@v4
# Downloads your repository code to the runner
- name: Set up Python 3.11
uses: actions/setup-python@v4
with:
python-version: '3.11'
```

```
cache: 'pip'
# Caches pip packages to speed up future runs
- name: Install dependencies
run: |
python -m pip install --upgrade pip
pip install -r app/requirements.txt
- name: Run pytest
run: |
cd app
pytest tests/ -v --tb=short
# -v = verbose output
# --tb=short = shorter traceback on failure
- name: Upload test results
if: always() # Run even if tests fail
run: echo "Tests completed — check output above for results"
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# JOB 2: BUILD
# Build Docker image — only runs if all tests passed
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
build:
name: Build Docker Image
runs-on: ubuntu-latest
needs: test # Only starts if 'test' job succeeded
outputs:
image_tag: ${{ steps.meta.outputs.version }}
steps:
- name: Checkout code
uses: actions/checkout@v4
- name: Set up Docker Buildx
uses: docker/setup-buildx-action@v3
# Enables advanced Docker build features (caching, multi-platform)
- name: Extract metadata for Docker image
id: meta
run: |
echo "version=${GITHUB_SHA::8}" >> $GITHUB_OUTPUT
# Uses first 8 characters of git commit hash as image tag
# Example: a1b2c3d4
- name: Build Docker image
run: |
docker build -t ${{ env.DOCKER_IMAGE }}:${{ steps.meta.outputs.version }} .
docker build -t ${{ env.DOCKER_IMAGE }}:latest .
echo "Image built successfully: ${{ env.DOCKER_IMAGE }}:${{ steps.meta.outputs.version }}"
- name: Save image to cache
uses: actions/cache@v3
with:
path: /tmp/docker-image.tar
key: docker-${{ github.sha }}
# Saves the built image so the next job can use it without rebuilding
```

```
- name: Export image
run: docker save ${{ env.DOCKER_IMAGE }}:latest > /tmp/docker-image.tar
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■
# JOB 3: PUSH
# Push Docker image to Docker Hub
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■
push:
name: Push to Docker Hub
runs-on: ubuntu-latest
needs: build # Only starts if 'build' job succeeded
steps:
- name: Checkout code
uses: actions/checkout@v4
- name: Set up Docker Buildx
uses: docker/setup-buildx-action@v3
- name: Login to Docker Hub
uses: docker/login-action@v3
with:
username: ${{ secrets.DOCKERHUB_USERNAME }}
password: ${{ secrets.DOCKERHUB_TOKEN }}
# These come from GitHub repository secrets — never hardcode credentials!
- name: Build and push both tags
run: |
SHA_TAG="${GITHUB_SHA::8}"
docker build -t ${{ env.DOCKER_IMAGE }}:${SHA_TAG} -t ${{ env.DOCKER_IMAGE }}:latest .
docker push ${{ env.DOCKER_IMAGE }}:${SHA_TAG}
docker push ${{ env.DOCKER_IMAGE }}:latest
echo "Pushed: ${{ env.DOCKER_IMAGE }}:${SHA_TAG}"
echo "Pushed: ${{ env.DOCKER_IMAGE }}:latest"
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■
# JOB 4: DEPLOY
# Update Kubernetes deployment with new image
# Only runs on push to main (not on pull requests)
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■
deploy:
name: Deploy to Kubernetes
runs-on: self-hosted # Runs on YOUR machine where Minikube is running
needs: push # Only starts if 'push' job succeeded
if: github.event_name == 'push' && github.ref == 'refs/heads/main'
steps:
- name: Checkout code
uses: actions/checkout@v4
- name: Set image tag
id: tag
run: echo "sha=${GITHUB_SHA::8}" >> $GITHUB_OUTPUT
- name: Update Kubernetes deployment
```

```
run: |
kubectl set image deployment/${{ env.DEPLOYMENT_NAME }} \
${{ env.DEPLOYMENT_NAME }}=${{ env.DOCKER_IMAGE }}:${{ steps.tag.outputs.sha }}
echo "Deployment updated with image tag: ${{ steps.tag.outputs.sha }}"
- name: Wait for rollout to complete
run: |
kubectl rollout status deployment/${{ env.DEPLOYMENT_NAME }} --timeout=120s
echo "Rollout completed successfully!"
- name: Verify deployment
run: |
kubectl get pods
kubectl get services
echo "Deployment verified — new version is live!"
- name: Print success message
run: |
echo "==================================="
echo "DEPLOYMENT SUCCESSFUL!"
echo "Image: ${{ env.DOCKER_IMAGE }}:${{ steps.tag.outputs.sha }}"
echo "==================================="
```

5.2 — Self-Hosted Runner Setup Guide

The deploy job needs to run on YOUR machine because Minikube runs locally. A self-hosted runner is a small program that listens for GitHub Actions jobs and runs them on your computer.

1 **Go to your GitHub repository**

Repository → Settings → Actions → Runners → New self-hosted runner

2 **Select your OS**

Choose Linux, macOS, or Windows and follow the commands shown on screen

3 **Download runner (example for Linux)**

mkdir actions-runner && cd actions-runner && curl -o actions-runner-linux-x64-2.311.0.tar.gz -L https://github.com/actions/runner/releases/download/v2.311.0/actions-runner-linux-x64-2.311.0.tar.gz && tar xzf ./actions-runner-linux-x64-2.311.0.tar.gz

4 **Configure runner**

./config.sh --url https://github.com/YOUR_USERNAME/autodeploy --token YOUR_TOKEN_FROM_GITHUB

5 **Start runner as background service**

sudo ./svc.sh install && sudo ./svc.sh start

6 **Verify runner is online**

GitHub → Settings → Actions → Runners — should show your runner as Idle (green dot)

5.3 — Add GitHub Secrets

GitHub Secrets are encrypted variables your pipeline uses without exposing credentials in code. Go to: Repository → Settings → Secrets and variables → Actions → New repository secret

Secret Name	Value	Where to Find It
DOCKERHUB_USERNAME	Your Docker Hub username	hub.docker.com profile
DOCKERHUB_TOKEN	Your Docker Hub access token	Docker Hub → Account Settings → Security

- ■ deploy.yml committed to .github/workflows/
- ■ DOCKERHUB_USERNAME secret added to GitHub repository
- ■ DOCKERHUB_TOKEN secret added to GitHub repository
- ■ Self-hosted runner shows as Idle in GitHub Settings
- ■ Push a commit and watch the Actions tab — all 4 jobs should turn green
- ■ Docker Hub shows new image with SHA tag

SECTION 6 — Days 8–10: Prometheus + Grafana Monitoring

Add production-grade observability — metrics, dashboards, and alerting

Goal: Prometheus automatically collects metrics from your app every 15 seconds. Grafana displays them on a live dashboard showing request rate, latency, CPU, and memory. An alert fires automatically when a pod crashes.

■ AI PROMPT — Day 8 — Prometheus Stack Installation

I have Minikube running with my FastAPI app deployed. Install the kube-prometheus-stack using Helm. Give me exact commands to add the repo, install the stack, access Grafana in the browser via Minikube port-forward, and the default login credentials. Also show me how to verify Prometheus is running and accessible.

6.1 — Complete prometheus-values.yaml

■ monitoring/prometheus-values.yaml

```
# Helm values for kube-prometheus-stack
# This customizes the default Prometheus + Grafana installation
grafana:
# Change default admin password (use a strong password in production)
adminPassword: "autodeploy-admin"
# Grafana service configuration
service:
type: NodePort # Expose Grafana via Minikube NodePort
nodePort: 30300 # Access on port 30300
# Pre-configure Prometheus as a data source
additionalDataSources:
- name: Prometheus
type: prometheus
url: http://prometheus-operated:9090
access: proxy
isDefault: true
prometheus:
prometheusSpec:
# How long to keep metrics data
retention: 7d
# Resource limits for Prometheus pod
resources:
requests:
memory: "256Mi"
cpu: "100m"
limits:
memory: "512Mi"
cpu: "200m"
```

```
# Which ServiceMonitors to pick up (match all namespaces)
serviceMonitorSelectorNilUsesHelmValues: false
serviceMonitorSelector: {}
serviceMonitorNamespaceSelector: {}
alertmanager:
# Keep Alertmanager simple for local development
alertmanagerSpec:
resources:
requests:
memory: "64Mi"
cpu: "50m"
limits:
memory: "128Mi"
cpu: "100m"
```

6.2 — Complete servicemonitor.yaml

■ k8s/servicemonitor.yaml

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
name: autodeploy-monitor
labels:
# This label tells Prometheus to pick up this ServiceMonitor
release: prometheus
spec:
selector:
matchLabels:
app: autodeploy # Match our app's service label
endpoints:
- port: http # Port name from service.yaml
path: /metrics # Prometheus scrape endpoint (we add this to FastAPI)
interval: 15s # Scrape metrics every 15 seconds
```

6.3 — Updated main.py with Prometheus Metrics

■ app/main.py (updated — add these lines)

```
# Add this import at the top of main.py
from prometheus_fastapi_instrumentator import Instrumentator
# Add these lines right after app = FastAPI(...)
# This adds automatic HTTP metrics collection to every endpoint
instrumentator = Instrumentator(
should_group_status_codes=False,
should_ignore_untemplated=True,
should_respect_env_var=True,
should_instrument_requests_inprogress=True,
excluded_handlers=["/metrics"],
inprogress_name="http_requests_inprogress",
```

```
inprogress_labels=True,
)
instrumentator.instrument(app).expose(app)
# This automatically adds a GET /metrics endpoint that Prometheus scrapes
# Metrics included: request count, request duration, in-progress requests
```

6.4 — Installation Commands

```
# Step 1: Add Prometheus Helm repository
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
# Step 2: Create monitoring namespace
kubectl create namespace monitoring
# Step 3: Install kube-prometheus-stack with our custom values
helm install prometheus prometheus-community/kube-prometheus-stack \
--namespace monitoring \
--values monitoring/prometheus-values.yaml
# Step 4: Wait for all pods to be ready (takes 2-3 minutes)
kubectl get pods -n monitoring --watch
# Step 5: Access Grafana in browser
minikube service prometheus-grafana --namespace monitoring --url
# OR use port-forward:
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80
# Then open: http://localhost:3000
# Login: admin / autodeploy-admin
# Step 6: Apply ServiceMonitor
kubectl apply -f k8s/servicemonitor.yaml
# Step 7: Verify Prometheus is scraping your app
# Go to Prometheus UI:
kubectl port-forward -n monitoring svc/prometheus-operated 9090:9090
# Open: http://localhost:9090/targets
# Your app should appear as a target with "UP" status
```

Grafana Dashboard — PromQL Queries

Panel Name	PromQL Query	Visualization
HTTP Request Rate	rate(http_requests_total[5m])	Time series graph
P95 Request Latency	histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))	Time series graph
Pod CPU Usage	rate(container_cpu_usage_seconds_total{pod=~'autodeploy.*'}[5m])	Gauge
Pod Memory Usage	container_memory_usage_bytes{pod=~'autodeploy.*'}	Gauge
Active Requests	http_requests_inprogress	Stat panel
Error Rate	rate(http_requests_total{status=~'5..'}[5m])	Time series graph

■ kubectl get pods -n monitoring shows all pods Running

■ Grafana login works at http://localhost:3000 (admin/autodeploy-admin)

■ Prometheus targets page shows autodeploy app as UP

■ /metrics endpoint on your FastAPI app returns Prometheus metrics

■ Grafana dashboard shows live request rate data

■ Alert rule created for pod CrashLoopBackOff

SECTION 7 — Days 11–14: Polish, README & Resume

Make your project portfolio-ready and interview-proof

■ AI PROMPT — Day 11 — README Generation

Write a professional GitHub README.md for my AutoDeploy project. Include: title with badges, one-line description, Mermaid architecture diagram, tech stack table, 6 key features, prerequisites, 5-command quick start, CI/CD pipeline explanation, monitoring section, project structure tree, and author section. Make it impressive enough to get noticed by a DevOps hiring manager in India.

7.1 — Complete README.md

■ README.md

```
# ■ AutoDeploy — Production CI/CD Pipeline
[![CI/CD Pipeline](https://github.com/YOUR_USERNAME/autodeploy/workflows/AutoDeploy%20CI%2FCD%20Pipeline/badge.svg)](https://github.com/YOUR_USERNAME/autodeploy/actions)
[![Docker](https://img.shields.io/badge/Docker-Ready-blue?logo=docker)](https://hub.docker.com/r/YOUR_USERNAME/autodeploy)
[![Kubernetes](https://img.shields.io/badge/Kubernetes-Deployed-326CE5?logo=kubernetes)](https://kubernetes.io)
[![Python](https://img.shields.io/badge/Python-3.11-green?logo=python)](https://python.org)
[![License: MIT](https://img.shields.io/badge/License-MIT-yellow)](LICENSE)
> A production-grade CI/CD pipeline that automatically tests, builds, and deploys
> a Python FastAPI app to Kubernetes on every git push — with live monitoring.
---
## ■■ Architecture
```
GitHub Push
■
▼
■■■
■ GitHub Actions Pipeline ■
■ ■■■■■■■■■■ ■■■■■■■■■■■ ■■■■■■■■ ■■■■■■■■■■■■ ■
■ ■ TEST ■■■■ BUILD ■■■■ PUSH ■■■■ DEPLOY ■ ■
■ ■pytest ■ ■ Docker ■ ■Docker■ ■ kubectl ■ ■
■ ■■■■■■■■■■ ■■■■■■■■■■■ ■ Hub ■ ■■■■■■■■■■■■ ■
■■■
■
▼
■■■■■■■■■■■■■■■■■■
■ Minikube K8s ■
■ 2x Replicas ■
■ AutoDeploy ■
■■■■■■■■■■■■■■■■■■
■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
▼ ▼
```
```

Features

- **Automated CI/CD**: 4-stage pipeline runs on every git push
- **Zero-downtime deploys**: Kubernetes rolling updates with instant rollback
- **Containerized**: Docker multi-stage build for minimal image size
- **Self-healing**: K8s liveness probes auto-restart failed pods
- **Live monitoring**: Prometheus + Grafana dashboard with alerting
- **Fully free**: No cloud costs — runs entirely on local Minikube

Tech Stack

Tool	Purpose
Python FastAPI	REST API web application
Docker	Container build & packaging
Docker Hub	Container image registry
Minikube	Local Kubernetes cluster
GitHub Actions	CI/CD pipeline automation
Prometheus	Metrics collection
Grafana	Metrics visualization

Quick Start

```bash
# 1. Clone the repository
git clone https://github.com/YOUR_USERNAME/autodeploy.git && cd autodeploy

# 2. Start Minikube
minikube start --driver=docker

# 3. Deploy to Kubernetes
kubectl apply -f k8s/

# 4. Get the app URL
minikube service autodeploy-service --url

# 5. Open in browser
# Navigate to: [URL from step 4]/health
```

How the Pipeline Works

1. **Push** code to the `main` branch on GitHub
2. **Test** job runs `pytest` — pipeline stops if any test fails
3. **Build** job creates a Docker image tagged with git commit SHA
4. **Push** job uploads image to Docker Hub (`:latest` and `:[SHA]` tags)
5. **Deploy** job runs `kubectl set image` to update the K8s deployment
6. Kubernetes performs a **rolling update** — zero downtime

Monitoring

Access the Grafana dashboard:

```bash
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80
# Open: http://localhost:3000 (admin / autodeploy-admin)
```

Dashboard panels: HTTP request rate, P95 latency, pod CPU, pod memory.

```
## ■ Project Structure
```
```
autodeploy/
■■■ app/ # FastAPI application
■ ■■■ main.py
■ ■■■ requirements.txt
■ ■■■ tests/test_main.py
■■■ k8s/ # Kubernetes manifests
■ ■■■ deployment.yaml
■ ■■■ service.yaml
■ ■■■ ingress.yaml
■■■ monitoring/ # Prometheus + Grafana config
■ ■■■ prometheus-values.yaml
■■■ .github/workflows/ # CI/CD pipeline
■ ■■■ deploy.yml
■■■ Dockerfile
```
```
## ■ Author
**Pravinkumar Choudhary**
- GitHub: [@speedprav](https://github.com/speedprav)
- LinkedIn: [pravin-choudhary](https://linkedin.com/in/pravin-choudhary-427ab9233)
- BCA · Cloud Computing · Parul University
```

7.2 — Resume Bullets (Copy-Paste Ready)

Bullet 1 — CI/CD Pipeline

> • Built 4-stage automated CI/CD pipeline using GitHub Actions, Docker, and Kubernetes — reduced deployment time from manual 20 min to zero-touch 3 min on every git push

Bullet 2 — Kubernetes

> • Deployed Python FastAPI application to Kubernetes (Minikube) with rolling updates, auto-rollback on failure, liveness/readiness probes, and horizontal pod scaling (2–4 replicas)

Bullet 3 — Monitoring

> • Implemented production observability stack using Prometheus and Grafana with custom FastAPI metrics endpoint, 6-panel live dashboard, and automated pod crash alerting

30-Second Verbal Pitch

> "I built AutoDeploy, a production CI/CD pipeline where every git push automatically runs tests, builds a Docker container, pushes it to a registry, and deploys it to Kubernetes — all in under 3 minutes with zero manual steps. I added Prometheus and Grafana for live monitoring with alerts. The whole thing runs free using Minikube and GitHub Actions. It demonstrates the exact DevOps workflow used in professional engineering teams."

SECTION 8 — All AI Prompts Quick Reference

Every prompt in one place — copy and paste into Claude or ChatGPT

■ AI PROMPT — Day 0 — Prerequisites Setup

I am setting up my development environment for a DevOps project on [Windows/Mac/Linux]. I need to install: Docker Desktop, Minikube, kubectl, Python 3.10+, Git, and VS Code. Give me step-by-step installation commands for my OS, how to verify each tool works, and common errors to watch for during installation.

■ AI PROMPT — Day 1 — FastAPI Application

I am building a DevOps portfolio project called AutoDeploy. Generate a Python FastAPI app with: GET /health (status+timestamp), GET /info (project metadata), POST /predict (mock NLP). Include complete main.py, requirements.txt, and tests/test_main.py with pytest. Add detailed comments explaining every line.

■ AI PROMPT — Day 2 — Dockerfile + Docker Compose

I have a Python FastAPI app. Give me: a production Dockerfile using python:3.11-slim with multi-stage build and non-root user, a docker-compose.yml, a .dockerignore, and step-by-step commands to build/run/test/push to Docker Hub as [USERNAME]/autodeploy.

■ AI PROMPT — Day 3 — Kubernetes Manifests

I have Docker image [USERNAME]/autodeploy:latest on Docker Hub. Give me complete Kubernetes YAML files: deployment.yaml (2 replicas, resource limits, liveness+readiness probes), service.yaml (NodePort), ingress.yaml. Include kubectl commands to deploy and get the app URL. Explain each Kubernetes concept in simple terms.

■ AI PROMPT — Day 4 — Kubectl Operations

I have a Kubernetes deployment running in Minikube. Give me exact kubectl commands and explanations for: updating to new image version, watching rolling update in real time, rolling back to previous version, scaling from 2 to 4 replicas and back, viewing pod logs.

■ AI PROMPT — Day 5 — GitHub Actions Pipeline

Generate a complete .github/workflows/deploy.yml with 4 sequential jobs: test (pytest), build (docker build), push (docker push to Docker Hub with :latest and :SHA tags), deploy (kubectl set image on self-hosted runner). Jobs use 'needs:' for sequencing. Use secrets DOCKERHUB_USERNAME and DOCKERHUB_TOKEN. Deploy only on main branch push.

■ AI PROMPT — Day 6 — Self-Hosted Runner

I need a self-hosted GitHub Actions runner on [Windows/Mac/Linux] to deploy to Minikube. Give me: step-by-step runner registration, how to run it as a background service, how to update deploy.yml for 'runs-on: self-hosted' on deploy job only, how to verify it's working, and common errors with fixes.

■ AI PROMPT — Day 7 — Pipeline End-to-End Test

My GitHub Actions CI/CD pipeline is set up. Walk me through a complete end-to-end test: what change to make, how to watch pipeline run on GitHub, verify image on Docker Hub, confirm new version in Minikube, intentionally break a test to verify failure stops pipeline, then fix and re-trigger. Explain like a senior DevOps engineer coaching a junior.

■ AI PROMPT — Day 8 — Prometheus + Grafana Install

I have Minikube running with my FastAPI app deployed. Give me commands to: install Helm, add prometheus-community Helm repo, install kube-prometheus-stack with custom values, access Grafana via Minikube, get default login credentials, verify Prometheus is scraping metrics.

■ AI PROMPT — Day 9 — FastAPI Prometheus Metrics

I have FastAPI running in Kubernetes and Prometheus installed. Show me how to add prometheus-fastapi-instrumentator to expose /metrics endpoint, create a ServiceMonitor YAML so Prometheus auto-discovers my app, rebuild and redeploy, verify metrics appear in Prometheus UI. Show the exact metric names I should look for.

■ AI PROMPT — Day 10 — Grafana Dashboard + Alerts

Prometheus is scraping my FastAPI app. Create a Grafana dashboard with 4 panels: HTTP request rate, P95 latency, pod CPU usage, pod memory. Give me the exact PromQL queries. Also set up an alert rule for CrashLoopBackOff and show how to export dashboard JSON for GitHub.

■ AI PROMPT — Day 11 — README.md

Write a professional GitHub README.md for AutoDeploy: Python FastAPI + Docker + Kubernetes (Minikube) + GitHub Actions + Prometheus + Grafana. Include: title with badges, Mermaid architecture diagram, tech stack table, 6 features, prerequisites, 5-command quick start, pipeline explanation, monitoring section, project tree, author section with LinkedIn+GitHub.

■ AI PROMPT — Day 13 — Code Review + Cleanup

Review my AutoDeploy project files: [paste main.py, deploy.yml, deployment.yaml]. Check for: security issues (hardcoded creds, exposed ports), K8s and Docker best practice violations, missing error handling, performance improvements. Give improved version with explanations for every change.

■ AI PROMPT — Day 14 — Resume + LinkedIn

I completed AutoDeploy: FastAPI + Docker + GitHub Actions (4 jobs) + Kubernetes (Minikube) + Prometheus + Grafana. Give me: 3 resume bullet points (strong verbs, specific numbers), LinkedIn post (150 words), 3 interview questions with model answers, 30-second verbal pitch. I am a BCA student targeting Cloud/DevOps internships in India.

■ AI PROMPT — Any Day — Universal Error Fix

I am building AutoDeploy on Day [X]. I ran this command: [paste command]. I got this error: [paste full error message]. My OS is [Windows/Mac/Linux]. Here is my relevant file: [paste file]. Please: 1) explain what this error means, 2) give me the exact fix with commands, 3) tell me how to verify the fix worked.

SECTION 9 — Troubleshooting Guide

Most common errors, what they mean, and exactly how to fix them

■ ImagePullBackOff

Cause: Kubernetes cannot download the Docker image

Fix: 1. Check image name: kubectl describe pod POD_NAME 2. Verify image exists on Docker Hub 3. Check spelling of username/image in deployment.yaml 4. Make Docker Hub repo is public

■ CrashLoopBackOff

Cause: Pod starts then immediately crashes, Kubernetes keeps restarting it

Fix: 1. Check logs: kubectl logs POD_NAME --previous 2. Usually a Python error in your app code 3. Test locally: docker run YOUR_IMAGE and check output 4. Verify requirements.txt has all dependencies

■ Connection refused / Cannot access app URL

Cause: Service not exposed correctly or Minikube networking issue

Fix: 1. Run: minikube service autodeploy-service --url 2. Verify service: kubectl get services 3. Check pod is Running: kubectl get pods 4. Try: minikube tunnel (in separate terminal)

■ Pipeline: Error: kubectl command not found

Cause: kubectl not available on GitHub Actions runner

Fix: 1. Confirm deploy job uses 'runs-on: self-hosted' 2. Verify kubectl installed: kubectl version --client 3. Check runner is online in GitHub Settings → Actions → Runners 4. Restart runner service

■ Pipeline: denied: access forbidden (Docker Hub push)

Cause: Wrong Docker Hub credentials in GitHub Secrets

Fix: 1. Go to Docker Hub → Account Settings → Security 2. Delete old token and create a new Access Token 3. Update DOCKERHUB_TOKEN secret in GitHub repo settings 4. Ensure DOCKERHUB_USERNAME is your exact login username

■ Prometheus shows no targets / app not scraped

Cause: ServiceMonitor not configured correctly

Fix: 1. Check ServiceMonitor: kubectl get servicemonitor 2. Verify label 'release: prometheus' on ServiceMonitor 3. Check /metrics returns data: curl http://APP_URL/metrics 4. Re-apply: kubectl apply -f k8s/servicemonitor.yaml

■ Grafana shows No Data on panels

Cause: Prometheus data source not connected or wrong PromQL query

Fix: 1. Grafana → Configuration → Data Sources → Test Prometheus 2. Go to Prometheus UI → Graph → paste your PromQL query 3. Check metric names at: http://localhost:9090/metrics 4. Verify your app has received at least 1 request

■ Minikube start fails

Cause: Docker Desktop not running or insufficient resources

Fix: 1. Open Docker Desktop and wait for 'Engine Running' 2. Delete and restart: minikube delete && minikube start --driver=docker 3. Increase resources: --cpus=2 --memory=4096 4. On Windows: Enable WSL2 integration in Docker Desktop settings

BONUS — Interview Questions & Model Answers

Be ready for every question a DevOps interviewer will ask about this project

Q: What is CI/CD and how does your pipeline implement it?

A: CI stands for Continuous Integration — every code push is automatically tested before merging, catching bugs early. CD stands for Continuous Deployment — code that passes all tests is automatically deployed without manual steps. In AutoDeploy, my pipeline has 4 jobs that run in sequence on every git push to main: first pytest runs all tests, then Docker builds the image, then it pushes to Docker Hub with both a latest tag and a unique commit SHA tag, then kubectl updates the Kubernetes deployment with the new image. If any job fails, the pipeline stops immediately and nothing gets deployed. This eliminates human error and ensures only tested code ever reaches the running application.

Q: What is Kubernetes and why use it instead of just Docker?

A: Docker runs individual containers but has no intelligence about failures or scaling. If a Docker container crashes, it stays down. Kubernetes is a container orchestration system that adds: automatic restarts when containers crash (self-healing), rolling updates that deploy new versions with zero downtime by gradually replacing old pods, horizontal scaling where you can go from 2 to 10 replicas with one command, load balancing across all replicas, and health monitoring via liveness and readiness probes. In my project, I configured 2 replicas so if one pod crashes, traffic automatically routes to the other while Kubernetes restarts the crashed pod — this would be impossible with plain Docker.

Q: What happens if your deployment fails mid-rollout?

A: Kubernetes handles this automatically with my configuration. I set maxUnavailable to 0 in the rolling update strategy, which means Kubernetes never takes a pod offline until the new pod is confirmed healthy by the readiness probe. If the new pod fails its readiness check, Kubernetes stops the rollout and keeps the old pods running — so users never see downtime. I can also manually trigger a rollback with kubectl rollout undo deployment/autodeploy, which instantly reverts to the previous known-good image. The rollout history command shows all previous deployments, so I can roll back to any specific version.

Q: How does your monitoring stack work?

A: I use a three-layer approach. First, my FastAPI app exposes a /metrics endpoint using prometheus-fastapi-instrumentator, which automatically tracks HTTP request count, request duration histograms, and in-progress requests for every API endpoint. Second, Prometheus scrapes this endpoint every 15 seconds, configured via a ServiceMonitor Kubernetes resource that automatically discovers my app. Prometheus stores all metrics in a time-series database. Third, Grafana connects to Prometheus as a data source and displays dashboards with panels showing request rate, P95 latency, pod CPU, and pod memory using PromQL queries. I also configured an alert rule that fires when a pod enters CrashLoopBackOff state.

Q: What would you change if you used AWS instead of local tools?

A: The core concepts stay identical but the tools change. I would replace Minikube with AWS EKS for a managed Kubernetes control plane that AWS maintains. Docker Hub would be replaced by AWS ECR for the container registry, which integrates more securely with EKS using IAM roles. I would add Terraform to provision the infrastructure as code — VPC, subnets, EKS cluster, node groups. GitHub Actions would authenticate to AWS using OIDC federation instead of stored access keys, which is the secure modern approach. CloudWatch Container Insights would replace or supplement Prometheus for AWS-native monitoring. The GitHub Actions workflow structure and Kubernetes manifests would stay almost identical.

Q: What was the hardest part of this project?

A: The most challenging part was configuring the self-hosted GitHub Actions runner to have access to Minikube's kubeconfig. When GitHub Actions runs a job on a self-hosted runner, it starts in a clean environment that doesn't automatically have kubectl configured to talk to Minikube. I had to ensure the runner's environment had the correct KUBECONFIG environment variable pointing to the Minikube-generated config file, and that the runner service had the right permissions. Another challenge was understanding the difference between how Prometheus discovers targets — I initially had the wrong label selectors on my ServiceMonitor which meant Prometheus ignored it completely.

You Have Everything You Need ■

Follow the sections in order. Copy the code. Run the commands. Paste the prompts when you need help. Ship it to GitHub.

Pravinkumar Choudhary

github.com/speedprav · linkedin.com/in/pravin-choudhary-427ab9233